

EXPECTATION MAXIMIZATION ALGORITHM FOR RECOMMENDATION SYSTEM NETFLIX

Application: Collaborative Filtering in

Presenter

—

Arish Muthukumar
SC25M159

Course

—

MA625
Statistical Models & Analysis

Instructor

—

Dr. Deepak T G
Professor and Head

PROBLEM STATEMENT

Given partially observed user-movie rating matrix,
estimate:

1. Hidden user preference groups
2. Missing ratings
3. Probability that a user belongs to a preference cluster

Using Expectation Maximization, we iteratively estimate:

- Hidden user interests
- Rating distribution parameters

GOAL

Predict movies that a user is likely to watch or
rate highly.



SAMPLE DATASET



VEENUS

1

4

5

?

DIVYA

3

?

2

4

RAVI

4

2

?

5

UJJAL

?

5

4

?

SHREYA

5

3

1

4

Rating Scale: 1 – 5

? – Missing Rating

Problem: Predict missing entries and recommend movies.

LATENT VARIABLES

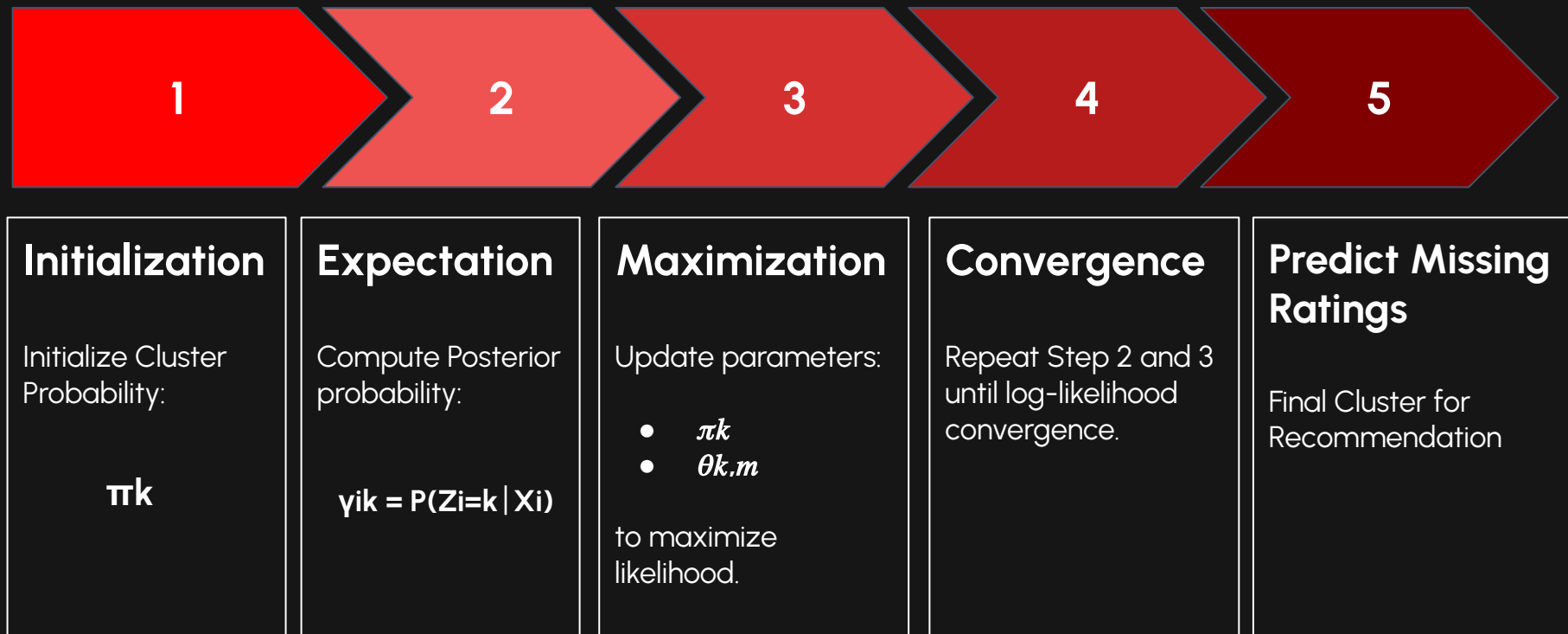
Let, Z_i = latent cluster for user i , $Z \in \{1,2\}$

CLUSTER	PREFERENCE
Z_1	Action Lovers
Z_2	Romance Lovers

- The clusters are not directly observed.
- EM is used to estimate them.

EM ALGORITHM

Basic for many unsupervised clustering algorithm.



STEP 1: INITIALIZATION

First replace missing values with column mean

M1 Mean

$$M1 = \frac{1 + 3 + 4 + 5}{4} = 3.25$$

M2 Mean

$$M2 = \frac{4 + 2 + 5 + 3}{4} = 3.5$$

M3 Mean

$$M3 = \frac{5 + 2 + 4 + 1}{4} = 3$$

M4 Mean

$$M4 = \frac{4 + 5 + 4}{3} = 4.33$$

USER	M1	M2	M3	M4
VEENUS	1	4	5	4.33
DIVYA	3	3.5	2	4
RAVI	4	2	3	5
UJJAL	3.25	5	4	4.33
SHREYA	5	3	1	4

STEP 1: INITIALIZATION

Python code for

- Create dataset
- Missing values(using Column Mean)

Create Dataset

```
data = pd.DataFrame({  
    "User": ["Veenus", "Divya", "Ravi", "Ujjal", "Shreya"],  
  
    "M1": [1,3,4,np.nan,5],  
    "M2": [4,np.nan,2,5,3],  
    "M3": [5,2,np.nan,4,1],  
    "M4": [np.nan,4,5,np.nan,4]  
  
})  
  
print(data)
```

	User	M1	M2	M3	M4
0	Veenus	1.0	4.0	5.0	NaN
1	Divya	3.0	NaN	2.0	4.0
2	Ravi	4.0	2.0	NaN	5.0
3	Ujjal	NaN	5.0	4.0	NaN
4	Shreya	5.0	3.0	1.0	4.0

Handle Missing Values (Initialization)

```
data_filled = data.copy()  
  
for col in ["M1", "M2", "M3", "M4"]:  
    data_filled[col] = data_filled[col].fillna(data_filled[col].mean())  
  
print(data_filled)
```

...	User	M1	M2	M3	M4
0	Veenus	1.00	4.0	5.0	4.333333
1	Divya	3.00	3.5	2.0	4.000000
2	Ravi	4.00	2.0	3.0	5.000000
3	Ujjal	3.25	5.0	4.0	4.333333
4	Shreya	5.00	3.0	1.0	4.000000

STEP 1: INITIALIZATION

Cluster probability,
Probability that a randomly chosen user belongs to cluster k .
 $\pi_1 = \pi_2 = 0.5$, $\sigma_1 = \sigma_2 = 1$

$\theta_{k,m}$ = expected rating of movie m by cluster k
 k = cluster, m = movie

CLUSTER	M1	M2	M3	M4
C1	4	2	4	5
C2	2	4	3	3

Interpretation:

- Cluster 1 $\rightarrow$ likes Action $\rightarrow \theta_1 = (4, 2, 4, 5)$
- Cluster 2 $\rightarrow$ likes Romance $\rightarrow \theta_2 = (2, 4, 3, 3)$

STEP 1: INITIALIZATION

Cluster probability,

Probability that a randomly chosen user belongs to cluster k .

$$\pi_1 = \pi_2 = 0.5, \sigma_1 = \sigma_2 = 1$$

$\theta_{k,m}$ = expected rating of movie m by cluster k

k = cluster, m = movie

```
# Convert to Matrix
X = data_filled[["M1","M2","M3","M4"]].values
N,D = X.shape # (5,4)
```

X

```
array([[1.      , 4.      , 5.      , 4.33333333],
       [3.      , 3.5     , 2.      , 4.      ],
       [4.      , 2.      , 3.      , 5.      ],
       [3.25    , 5.      , 4.      , 4.33333333],
       [5.      , 3.      , 1.      , 4.      ]])
```

```
# Initialize EM Parameters
K = 2

np.random.seed(0)

pi = np.ones(K)/K

mu = np.array([
    [4,2,4,5], # cluster 0 → high M1 & M3
    [2,4,3,3] # cluster 1 → high M2
])

sigma = np.ones(K)
```

STEP 2: EXPECTATION

Compute posterior probability user belongs to cluster

$\gamma_{ik} \rightarrow$ Membership weight matrix

We measure similarity using squared error likelihood

$$\gamma_{ik} = P(Z = k | X_i)$$

$$\gamma_{ik} = \frac{\pi_k e^{-\frac{\|x_i - \mu_k\|^2}{2}}}{\sum_{j=1}^2 \pi_j e^{-\frac{\|x_i - \mu_j\|^2}{2}}}$$

EXPECTATION

```
max_iter = 100
epsilon = 0.001
log_likelihood_old = 0

for iteration in range(max_iter):
    gamma = np.zeros((N,K))

    # E STEP
    for k in range(K):
        diff = X - mu[k]
        dist = np.sum(diff**2,axis=1)
        gamma[:,k] = pi[k]*np.exp(-dist/(2*sigma[k]))
    gamma = gamma/(gamma.sum(axis=1,keepdims=True)+1e-10)
```

γ_{ik} value after 1 iteration

gamma

```
array([[6.11792026e-13, 9.99999999e-01],
       [5.11277850e-03, 9.94887221e-01],
       [8.74419732e-03, 9.91255802e-01],
       [1.87959529e-08, 9.99999980e-01],
       [6.34966177e-01, 3.65033819e-01]])
```

STEP 2:

EXPECTATION

Yik value after 1st iteration

USER	CLUSTER 1	CLUSTER 2
VEENUS	6.11792026e-13	9.99999999e-01
DIVYA	5.11277850e-03	9.94887221e-01
RAVI	8.74419732e-03	9.91255802e-01
UJJAL	1.87959529e-08	9.99999980e-01
SHREYA	6.34966177e-01	3.65033819e-01

These are soft cluster memberships.

STEP 3: MAXIMIZATION

Update Cluster probability

$$\pi_k = \frac{1}{N} \sum \gamma_{ik}$$

Update Rating Expectation

$$\theta_{k,m} = \frac{\sum \gamma_{ik} x_{im}}{\sum \gamma_{ik}}$$

Nk # sum of γ_{ik}

array([0.64882317, 4.35117682])

Expected rating of movie m by cluster k

mu # $\theta_{k,m}$

array([[4, 2, 1, 4],
[2, 3, 3, 4]])

sigma

array([0.51454042, 1.47754239])

pi # Cluster probability

array([0.12976463, 0.87023536])

```
# M STEP
Nk = gamma.sum(axis=0)

for k in range(K):
    mu[k] = (gamma[:,k].reshape(-1,1)*X).sum(axis=0)/(Nk[k]+1e-10)
    diff = X-mu[k]
    sigma[k] = (gamma[:,k]*np.sum(diff**2,axis=1)).sum()/(Nk[k]*D+1e-10)
    pi[k] = Nk[k]/N
```

STEP 3: MAXIMIZATION

Repeat for all movies,

CLUSTER	M1	M2	M3	M4
C1	4	2	4	5
C2	2	4	3	3

Expected rating after 1st Iteration

CLUSTER	M1	M2	M3	M4
C1	4	2	1	4
C2	2	3	3	4

STEP 4: CONVERGENCE CHECK

Compute log likelihood

```
# LOG LIKELIHOOD

likelihood = np.zeros((N,K))

for k in range(K):
    diff = X - mu[k]
    dist = np.sum(diff**2,axis=1)
    likelihood[:,k] = pi[k]*np.exp(-dist/(2*sigma[k]))

total_likelihood = np.sum(likelihood,axis=1)
log_likelihood_new = np.sum(np.log(total_likelihood+1e-10))

print("Iteration:",iteration,
      " Log Likelihood:",log_likelihood_new)
```

$$\log L = \sum_i \log \left(\sum_k \pi_k f(X_i | \theta_k) \right)$$

$$f(x_i | \theta_k) = e^{-\frac{\|x_i - \mu_k\|^2}{2}}$$

STEP 4: CONVERGENCE CHECK

If $\text{Change} < \text{Threshold}$ Stop

Otherwise Repeat : E-Step → M-Step.

$$|L_{\text{new}} - L_{\text{old}}| < 0.001$$

- Usually < 10 iteration

```
# Convergence Check

if abs(log_likelihood_new - log_likelihood_old) < epsilon:

    print("\nAlgorithm Converged")
    break

log_likelihood_old = log_likelihood_new
```

```
... Iteration: 0 Log Likelihood: -11.195729406675316
    Iteration: 1 Log Likelihood: -11.194029364099363
    Iteration: 2 Log Likelihood: -11.192786759322235
    Iteration: 3 Log Likelihood: -11.191877825682141

    Algorithm Converged
```

STEP 5: PREDICT MISSING RATINGS

```
predicted = gamma @ mu
```

```
print(predicted)
```

```
[[2.         3.         3.         4.         ]
 [2.0102351  2.99488245 2.9897649  4.         ]
 [2.01750741 2.99124629 2.98249259 4.         ]
 [2.00000004 2.99999998 2.99999996 4.         ]
 [3.27053527 2.36473235 1.72946472 3.99999999]]
```



VEENUS

1

4

5

4

DIVYA

3

2.99

2

4

RAVI

4

2

2.98

5

UJJAL

2

5

4

4

SHREYA

5

3

1

4

STEP 5: FINAL CLUSTER FOR RECOMMENDATION

```
clusters = np.argmax(gamma,axis=1)

data_filled["Cluster"] = clusters

print(data_filled)
```

...	User	M1	M2	M3	M4	Cluster
0	Veenus	1.00	4.0	5.0	4.333333	1
1	Divya	3.00	3.5	2.0	4.000000	1
2	Ravi	4.00	2.0	3.0	5.000000	1
3	Ujjal	3.25	5.0	4.0	4.333333	1
4	Shreya	5.00	3.0	1.0	4.000000	0

CLUSTER 0 - Action Lovers

Users:

- Shreya

CLUSTER 1 - Romance Lovers

Users:

- Veenus
- Divya
- Ravi
- Ujjal

EM ALGORITHM PROS & CONS

Pros

1. Handles Missing Data
2. Finds Latent variables
3. Works well for Probabilistic models
4. Guaranteed Likelihood Improvement
5. Simple & Systematic Procedure



Cons

1. Sensitive to Initialization
2. Converges to local Maximum
3. Slow Converges
4. Algorithm collapsed into one dominant cluster due to small dataset, similar ratings



THANKS